DECOMPILE

Turn any YouTube video into production-ready software.

AMD Radeon AI Hackathon - Track 2

Private AI agents for development and local deployment. Submission specification, August 2026.

Executive summary

Decompile is a local-first autonomous software engineering studio. A user supplies a YouTube URL; specialized agents collect evidence, design a product, generate engineering artifacts, and package a downloadable repository. The product deliberately treats videos as source material for implementation, not just material for summaries.

Problem and solution

Videos contain valuable product and interface knowledge but leave teams with no trustworthy engineering handoff. Decompile creates that handoff: requirements, architecture, API contracts, application scaffolds, DevOps assets, quality plan, and documentation. Every project is inspectable while it is generated.

Application scenarios

Use Decompile to recreate a workflow shown in a tutorial, turn a product walkthrough into a planning starter, convert an internal demo into a delivery package, or accelerate discovery before a product spike.

Architecture

Next.js provides the immersive workspace. FastAPI owns project state, artifacts, local-memory search, and ZIP packaging. Source adapters retrieve transcripts and prepare visual evidence. Local model adapters isolate Qwen2.5-VL, BGE/FAISS, PaddleOCR, and vLLM ROCm or llama.cpp HIP from the API contract.

Agent design

Planner, Product Manager, Software Architect, UX Designer, Frontend Engineer, Backend Engineer, Documentation Engineer, DevOps Engineer, QA Engineer, and Repository Builder each have a named role, state, execution log, timing, and artifacts. The client polls a single project resource to render orchestration honestly.

Models and AMD optimization

No hosted inference API is required. The intended production runtime is vLLM ROCm or llama.cpp HIP on AMD Radeon hardware. Qwen2.5-VL handles visual interpretation; BGE creates local embeddings; FAISS indexes memory; PaddleOCR extracts interface text. The dashboard makes device, VRAM, HIP inference device, tokens/sec, latency, embeddings/sec, and vision latency visible.

Benchmark methodology

The API exposes a repeatable baseline for transcript processing, vision, embeddings, planning, repository generation, and total runtime. Hardware-specific claims must be reproduced by warming the model, recording five runs, reporting median timing, and capturing rocm-smi utilization and VRAM once per second.

Future work

Add persistent job storage, direct video frame sampling with yt-dlp and ffmpeg, model-specific token streaming, full rocm-smi probes, authenticated multi-user workspaces, and artifact diff review.

Benchmark baseline

Stage	Baseline
Transcript processing	1.8 s
Vision adapter	4.2 s
Embeddings	0.7 s
Planning	3.6 s
Repository generation	1.1 s
Total	11.4 s